Assignment No. 5

NAME: Nikita Sandip Patil
PRN NO: 125B2F005
SUBJECT: DATABASE MANAGEMENT SYSTEM
DIV: IT A

Aim: Write cursor code using control structures and exception handling.

Cursor in DBMS

Definition

A **cursor** is a database object used to **retrieve, manipulate, and process data row by row** from the result set of a SQL query. It provides a way to handle records individually instead of processing all rows at once.

Need of Cursor

SQL is a **set-oriented language**, which means it processes multiple rows simultaneously. However, in some situations, **row-by-row processing is required**, such as:

- Performing calculations on each row
- Applying conditional logic
- Generating customized output

In such cases, a cursor is used.

Working of Cursor (Steps)

The working of a cursor involves the following steps:

1. **Declare Cursor**
   Define the cursor with a SELECT query.
2. **Open Cursor**
   Execute the query and allocate memory.
3. **Fetch Data**
   Retrieve rows one by one into variables.
4. **Process Data**
   Perform required operations on each row.

5. **Close Cursor**
   Release the active result set.
6. **Deallocate Cursor**
   Free the memory occupied by the cursor.

Types of Cursor

1. **Implicit Cursor**
   a. Automatically created by the DBMS
   b. Used for simple queries like INSERT, UPDATE, DELETE
2. **Explicit Cursor**
   a. Created and controlled by the user
   b. Used for complex row-by-row processing

Advantages

- Enables row-by-row processing
- Useful for complex business logic
- Provides better control over data handling

Disadvantages

- Slower than set-based operations
- Consumes more memory and resources
- Not efficient for large datasets

- **cursor code using control structures and exception handling.**

Enter password: *****

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 16

Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+------------------------+
| Database               |
+------------------------+
| airplanedb             |
| company_db             |
| companydb              |
| emp                    |
| employee               |
| employee_appraisal     |
| employee_db            |
| information_schema     |
| mysql                  |
| performance_schema     |
| sakila                 |
| smart_notekeeper       |
| sql_detective          |
| sys                    |
| world                  |
+------------------------+
15 rows in set (0.00 sec)

mysql> CREATE DATABASE company;
Query OK, 1 row affected (0.01 sec)

mysql> USE company;
Database changed
mysql> CREATE TABLE employees (
    -> emp_id INT PRIMARY KEY,
    -> name VARCHAR(50),
```

```
   ->    salary INT,
   ->    rating INT,
   ->    bonus_category  VARCHAR(30)
   -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE DATABASE employee_db;
ERROR 1007 (HY000): Can't create database 'employee_db'; database exists
mysql> USE employee_db;
Database changed
mysql> CREATE TABLE employees (
   ->    emp_id INT PRIMARY KEY,
   ->    name VARCHAR(50),
   ->    salary INT,
   ->    rating INT,
   ->    bonus_category  VARCHAR(30)
   -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO employees VALUES
   -> (101, 'Nikita', 50000, 5, NULL),
   -> (102, 'Vedant', 40000, 4, NULL),
   -> (103, 'Raj', 30000, 3, NULL),
   -> (104, 'Simran', 35000, 2, NULL);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM employees;
+----------+----------+----------+----------+-------------------+
| emp_id | name   | salary | rating | bonus_category |
+----------+----------+----------+----------+-------------------+
|   101 | Nikita  |  50000 |   5 | NULL            |
```

```
|   102 | Vedant   |  40000 | 4 | NULL           |
|   103 | Raj   |  30000 |     3 | NULL           |
|   104 | Simran |  35000 |     2 | NULL           |
+----------+----------+----------+----------+-------------------+
4 rows in set (0.00 sec)

mysql>
mysql> CREATE DATABASE IF NOT EXISTS employee_db;
Query OK, 1 row affected, 1 warning (0.00 sec)

mysql>
mysql> CREATE DATABASE IF NOT EXISTS employee_db;
Query OK, 1 row affected, 1 warning (0.00 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE appraisal_cursor()
   -> BEGIN
   ->    DECLARE done INT DEFAULT 0;
   ->    DECLARE v_id INT;
   ->    DECLARE v_salary INT;
   ->    DECLARE v_rating INT;
   ->    DECLARE v_category VARCHAR(30);
   ->
   ->    DECLARE emp_cursor CURSOR FOR
   ->    SELECT emp_id, salary, rating FROM employees;
   ->
   ->    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
   ->    OPEN emp_cursor;
   ->    read_loop: LOOP
   ->       FETCH emp_cursor INTO v_id, v_salary, v_rating;
   ->       IF done = 1 THEN
```

```
    ->         LEAVE read_loop;
    ->       END IF;
    ->       IF v_rating = 5 THEN
    ->         SET v_salary = v_salary + 10000;
    ->         SET v_category = 'Excellent';
    ->       ELSEIF v_rating = 4 THEN
    ->         SET v_salary = v_salary + 5000;
    ->         SET v_category = 'Good';
    ->       ELSEIF v_rating = 3 THEN
    ->         SET v_salary = v_salary + 2000;
    ->         SET v_category = 'Average';
    ->       ELSE
    ->         SET v_category = 'Needs Improvement';
    ->       END IF;
    ->       UPDATE employees
    ->       SET salary = v_salary,
    ->         bonus_category = v_category
    ->       WHERE emp_id = v_id;
    ->     END LOOP;
    ->     CLOSE emp_cursor;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL appraisal_cursor();
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM employees;
+----------+----------+----------+----------+----------------------+
| emp_id | name   | salary | rating | bonus_category    |
+----------+----------+----------+----------+----------------------+
```

```
|   101 | Nikita   |  60000 |   5 | Excellent           |
|   102 | Vedant   |  45000 | 4 | Good              |
|   103 | Raj    |  32000 |     3 | Average            |
|   104 | Simran |  35000 |     2 | Needs Improvement |
+----------+----------+----------+----------+----------------------+
4 rows in set (0.00 sec)

mysql> UPDATE employees SET
   -> salary = CASE
   ->    WHEN emp_id = 101 THEN 50000
   ->    WHEN emp_id = 102 THEN 40000
   ->    WHEN emp_id = 103 THEN 30000
   ->    WHEN emp_id = 104 THEN 35000
   -> END,
   -> bonus_category = NULL;
Query OK, 4 rows affected (0.00 sec)
Rows matched: 4 Changed: 4 Warnings: 0

mysql> UPDATE employees
   -> SET
   -> salary = salary +
   -> CASE
   ->    WHEN rating = 5 THEN 10000
   ->    WHEN rating = 4 THEN 5000
   ->    WHEN rating = 3 THEN 2000
   ->    ELSE 0
   -> END,
   -> bonus_category =
   -> CASE
   ->    WHEN rating = 5 THEN 'Excellent'
   ->    WHEN rating = 4 THEN 'Good'
   ->    WHEN rating = 3 THEN 'Average'
```

```
   ->    ELSE 'Needs Improvement'
   -> END;
Query OK, 4 rows affected (0.00 sec)
Rows matched: 4 Changed: 4 Warnings: 0

mysql> SELECT * FROM employees;
+----------+----------+----------+----------+----------------------+
| emp_id | name   | salary | rating | bonus_category    |
+----------+----------+----------+----------+----------------------+
|   101 | Nikita   |  60000 |   5 | Excellent           |
|   102 | Vedant   |  45000 | 4 | Good              |
|   103 | Raj    |  32000 |     3 | Average            |
|   104 | Simran |  35000 |     2 | Needs Improvement |
+----------+----------+----------+----------+----------------------+
4 rows in set (0.00 sec)

mysql> CALL appraisal_cursor();
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM employees;
+----------+----------+----------+----------+----------------------+
| emp_id | name   | salary | rating | bonus_category    |
+----------+----------+----------+----------+----------------------+
|   101 | Nikita   |  70000 |   5 | Excellent           |
|   102 | Vedant   |  50000 | 4 | Good              |
|   103 | Raj    |  34000 |     3 | Average            |
|   104 | Simran |  35000 |     2 | Needs Improvement |
+----------+----------+----------+----------+----------------------+
4 rows in set (0.00 sec)

mysql> USE employee_db;
Database changed
```

```
mysql> DROP TABLE IF EXISTS employees_tax;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> DROP TABLE IF EXISTS tax_log;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql> CREATE TABLE employees_tax (
   ->    emp_id INT,
   ->    salary INT
   -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE tax_log (
   ->    emp_id INT,
   ->    tax_amount INT,
   ->    total_tax_collected  INT
   -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO employees_tax VALUES
   -> (101, 60000),
   -> (102, 80000),
   -> (103, 30000),
   -> (104, 70000);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE tax_calculation()
   -> BEGIN
   ->    DECLARE done INT DEFAULT 0;
```

```
    ->    DECLARE v_id INT;
    ->    DECLARE v_salary INT;
    ->    DECLARE v_tax INT;
    ->    DECLARE total_tax INT DEFAULT 0;
    ->    DECLARE emp_cursor CURSOR FOR
    ->    SELECT emp_id, salary FROM employees_tax;
    ->    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    ->    OPEN emp_cursor;
    ->    read_loop: LOOP
    ->      FETCH emp_cursor INTO v_id, v_salary;
    ->      IF done = 1 THEN
    ->        LEAVE read_loop;
    ->      END IF;
    ->      IF v_salary > 50000 THEN
    ->        SET v_tax = v_salary * 0.10;
    ->        SET total_tax = total_tax + v_tax;
    ->        INSERT INTO tax_log
    ->        VALUES (v_id, v_tax, total_tax);
    ->      END IF;
    ->    END LOOP;
    ->    CLOSE emp_cursor;
    -> END$$
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL tax_calculation();
Query OK, 0 rows affected (0.01 sec)

mysql> select * from Tax_log;
+---------+--------------+------------------------+
| emp_id  | tax_amount  | total_tax_collected  |
```

```
+----------+--------------+-------------------------+
|   101 |     6000 |           6000 |
|   102 |     8000 |          14000 |
|   104 |     7000 |          21000 |
+----------+--------------+-------------------------+
3 rows in set (0.00 sec)

mysql> DROP TABLE IF EXISTS employees_trigger;
Query OK, 0 rows affected, 1 warning (0.00 sec)

mysql>
mysql> CREATE TABLE employees_trigger (
   ->    emp_id INT,
   ->    name VARCHAR(50),
   ->    salary INT,
   ->    rating INT
   -> );
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER $$
mysql> CREATE PROCEDURE process_appraisal()
   -> BEGIN
   ->    DECLARE done INT DEFAULT 0;
   ->    DECLARE v_id INT;
   ->    DECLARE v_salary INT;
   ->    DECLARE v_rating INT;
   ->    DECLARE emp_cursor CURSOR FOR
   ->    SELECT emp_id, salary, rating FROM employees_trigger;
   ->    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
   ->    OPEN emp_cursor;
   ->    read_loop: LOOP
   ->       FETCH emp_cursor INTO v_id, v_salary, v_rating;
```

```
    ->
    ->      IF done = 1 THEN
    ->        LEAVE read_loop;
    ->      END IF;
    ->
    ->      IF v_rating = 5 THEN
    ->        SET v_salary = v_salary + 10000;
    ->      ELSEIF v_rating = 4 THEN
    ->        SET v_salary = v_salary + 5000;
    ->      END IF;
    ->
    ->      UPDATE employees_trigger
    ->      SET salary = v_salary
    ->      WHERE emp_id = v_id;
    ->
    ->    END LOOP;
    ->
    ->    CLOSE emp_cursor;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql>
mysql> DELIMITER $$
mysql> CREATE TRIGGER trg_after_insert
    -> AFTER INSERT ON employees_trigger
    -> FOR EACH ROW
    -> BEGIN
    ->    CALL process_appraisal();
    -> END$$
Query OK, 0 rows affected (0.01 sec)
```

```
mysql>
mysql> DELIMITER ;

mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_before_insert
   -> BEFORE INSERT ON employees_trigger
   -> FOR EACH ROW
   -> BEGIN
   ->    IF NEW.rating = 5 THEN
   ->       SET NEW.salary = NEW.salary + 10000;
   ->    ELSEIF NEW.rating = 4 THEN
   ->       SET NEW.salary = NEW.salary + 5000;
   ->    END IF;
   -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> DROP TRIGGER IF EXISTS trg_after_insert;
Query OK, 0 rows affected (0.00 sec)

mysql> DROP PROCEDURE IF EXISTS process_appraisal;
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_before_insert
   -> BEFORE INSERT ON employees_trigger
   -> FOR EACH ROW
   -> BEGIN
   ->    IF NEW.rating = 5 THEN
```

```
    ->      SET NEW.salary = NEW.salary + 10000;
    ->    ELSEIF NEW.rating = 4 THEN
    ->      SET NEW.salary = NEW.salary + 5000;
    ->    END IF;
    -> END$$
ERROR 1359 (HY000): Trigger already exists
mysql>
mysql> DELIMITER ;
mysql> DELETE FROM employees_trigger;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO employees_trigger VALUES (201, 'Ravi', 40000, 5);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM employees_trigger;
+----------+-------+----------+----------+
| emp_id | name | salary | rating |
+----------+-------+----------+----------+
|  201 | Ravi |  50000 |     5 |
+----------+-------+----------+----------+
1 row in set (0.00 sec)
mysql> SELECT * FROM employees;
+----------+----------+----------+----------+----------------------+
| emp_id | name   | salary | rating | bonus_category    |
+----------+----------+----------+----------+----------------------+
|    101 | Nikita   |  70000 |   5 | Excellent            |
|   102 | Vedant   |  50000 | 4 | Good                |
|    103 | Raj    |  34000 |     3 | Average             |
|   104 | Simran |  35000 |     2 | Needs Improvement |
+----------+----------+----------+----------+----------------------+
4 rows in set (0.00 sec)
```

```
mysql> DELETE FROM employees;
Query OK, 4 rows affected (0.00 sec)

mysql> INSERT INTO employees (emp_id, salary) VALUES
   -> (101, 60000),
   -> (102, 45000),
   -> (103, 70000),
   -> (104, 30000);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO employees (emp_id, salary) VALUES
   -> (201, 60000),
   -> (202, 45000),
   -> (203, 70000),
   -> (204, 30000);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> DROP TABLE IF EXISTS employees_tax;
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE employees_tax (
   ->    emp_id INT PRIMARY KEY,
   ->    salary INT
   -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO employees_tax VALUES
   -> (101, 60000),
   -> (102, 45000),
   -> (103, 70000),
```

```
   -> (104, 30000);
Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> select * from employeeS;
+----------+-------+----------+----------+-------------------+
| emp_id | name | salary | rating | bonus_category |
+----------+-------+----------+----------+-------------------+
|  101 | NULL |  60000 |  NULL | NULL          |
|  102 | NULL |  45000 |  NULL | NULL          |
|  103 | NULL |  70000 |  NULL | NULL          |
|  104 | NULL |  30000 |  NULL | NULL          |
|  201 | NULL |  60000 |  NULL | NULL          |
|  202 | NULL |  45000 |  NULL | NULL          |
|  203 | NULL |  70000 |  NULL | NULL          |
|  204 | NULL |  30000 |  NULL | NULL          |
+----------+-------+----------+----------+-------------------+
8 rows in set (0.00 sec)

mysql> SELECT * FROM EMPLOYERE_tax;
ERROR 1146 (42S02): Table 'employee_db.employere_tax' doesn't exist
mysql> select * from employee_tax;
ERROR 1146 (42S02): Table 'employee_db.employee_tax' doesn't exist
mysql> select * from Employees_tax;
+----------+----------+
| emp_id | salary |
+----------+----------+
|  101 |  60000 |
|  102 |  45000 |
|  103 |  70000 |
|  104 |  30000 |
+----------+----------+
```

4 rows in set (0.00 sec)